Secure Hospital Management System

Project 3 Final Report - CYBS 3307 – Database Security

University of Hafr Al Batin | Term 2252

Prepared By:

Raim Bunaider Al-Dhafiry [redacted]

Submitted to: Dalal Almatrafi

Executive Summary

This report presents the design and security implementation of the Secure Hospital Management System, an Electronic Health Health Record (EHR) database solution intended to protect patient privacy and reduce unauthorized exposure of sensitive medical information.

The proposed system uses a structured relational database, Role-Based Access Control (RBAC), AES encryption for medical history and diagnoses, audit logs for accountability, and database replication for disaster recovery. The overall security model is aligned with the HIPAA Security Rule, which requires covered entities to protect electronic protected health information through administrative, physical, and technical safeguards.

Project Objectives

The primary objective of this project is to develop a **secure Electronic Health Record (EHR) system** to protect patient data privacy and support HIPAA-aligned safeguards.

The system distinguishes between basic patient data and highly sensitive clinical data, ensuring that each hospital staff member only accesses the information necessary for their specific role, supporting **confidentiality, integrity, and least-privilege access**.

Introduction: Healthcare Data Security & HIPAA

Hospitals store a combination of demographic information, medical history, diagnoses, prescriptions, and operational records. Because these records can identify a patient and reveal health conditions, they must be handled as highly sensitive information.

In the regulatory context, electronic protected health information (ePHI) is subject to the HIPAA Security Rule, which requires technical safeguards such as:

- Access Control
- Audit Controls
- Integrity Mechanisms
- Transmission Security

Project Requirements Mapping (Part 1)

Requirement	Design Decision	Security Benefit
RBAC for doctors and receptionists receptionists	Roles, permissions, role_permissions, MySQL users, grants, and column-level access.	Restricts each user to the minimum data needed for assigned duties.
Doctors cannot modify billing data data	Doctor role receives read-only billing access and no INSERT, INSERT, UPDATE, or DELETE privileges on billing.	Protects financial integrity and prevents unauthorized billing manipulation.
Receptionists see only basic patient details	Receptionist receives SELECT only on patient_id, full_name, full_name, phone, and address.	Prevents exposure of medical history, diagnosis, diagnosis, and treatment information.

Project Requirements Mapping (Part 2)

Requirement	Design Decision	Security Benefit
AES encryption for medical history history and diagnoses	Medical fields are stored using AES_ENCRYPT and retrieved retrieved with AES_DECRYPT only through authorized authorized access paths.	Reduces plaintext exposure if database rows are viewed or exported.
Audit logs to track patient data access	audit_logs table records user role, action, affected record, record, table name, timestamp, and IP address.	Supports accountability, investigations, and HIPAA HIPAA audit-control expectations.
Database replication for disaster recovery	Master-slave metadata tables and replication commands commands support standby recovery.	Improves availability and reduces downtime during during database failure.

Entity Classification & Data Types

Entity	Key Attributes	Data Type	Security Level
Patient	Name, Contact, DOB	VARCHAR, DATE	**Sensitive (PII)**
Doctor	Name, Specialization	VARCHAR	Internal
Medical Record	History, Diagnosis	VARBINARY (Encrypted)	**Highly Sensitive (ePHI)**
Billing	Amount, Status	DECIMAL, ENUM	Financial

Entity Relationship (ER) Diagram

Figure 1. ER Diagram showing secure hospital database schema with RBAC, encrypted medical records, audit logging, and replication entities.

The ER diagram shows a design consisting of twelve major entities: users, roles, permissions, role_permissions, patients, doctors, medical_records, billing, appointments, audit_logs, login_attempts, replication_master, and replication_slave. The users table connects each account to a role, while role_permissions resolves the many-to-many relationship between roles and permissions. The patients table represents basic patient information, and the medical_records table stores sensitive clinical content in encrypted fields. Billing is separated from medical records to ensure that clinical staff do not automatically receive modification privileges over financial records.

Entity	Purpose	Security Classification
users	Stores system accounts, password hashes, failed attempts, and active status.	Security entity
roles	Defines Doctor, Receptionist, and Admin roles.	Security entity
permissions	Defines resource-action permissions such as patients READ or billing UPDATE.	Security entity
role_permissions	Maps roles to permissions using a composite key.	RBAC junction table
patients	Stores basic demographic and contact information.	Core entity
medical_records	Stores encrypted diagnoses, treatment, history, and prescriptions.	Encrypted sensitive entity
billing	Stores payment and billing status linked to patients and records.	Financial entity
appointments	Stores patient visits and doctor scheduling.	Operational entity
audit_logs	Records access and modification activities.	Audit entity
login_attempts	Tracks successful and failed authentication events.	Security monitoring entity
replication_master	Stores master replication status metadata.	Disaster recovery entity
replication_slave	Stores standby server replication lag and relay-log metadata.	Disaster recovery entity

5. Role-Based Access Control Implementation

Role-Based Access Control is implemented through two complementary layers. The first layer is logical RBAC, where the roles, permissions, and role_permissions tables describe what each user type is allowed to do. The second layer is database-level enforcement, where MySQL users and GRANT statements limit access directly at the database engine. This layered design reduces risk because a user cannot bypass application logic and gain unrestricted database access.

Role	Patient Data	Medical Records	Billing	Appointments
Doctor	Read patient details	Read, create, and update authorized medical records	Read only; no modification	Read assigned appointments
Receptionist	Read basic columns only	No access	No access	Create, read, and update appointments
Admin	Full administrative management	Read encrypted records and manage system operations	Full management	Full management

The requirement that doctors cannot modify billing data is enforced by excluding INSERT, UPDATE, and DELETE billing permissions from the Doctor role. The requirement that receptionists can only see basic patient details is enforced using column-level SELECT access. The screenshot below demonstrates this approach by creating doctor_user and reception_user accounts, then granting the receptionist only selected patient columns.

Visualizing the core relationships between Patients, Doctors, Appointments, and Medical Records.

Role-Based Access Control (RBAC) Concept

Layer 1: Logical RBAC

Implemented through database tables (roles, permissions, and role_permissions). This layer defines what is permitted for each user type within the application logic.

Layer 2: Database Enforcement

Utilizes MySQL users and GRANT statements to restrict access directly at the database engine level. This ensures users cannot bypass application logic to gain unauthorized access.

RBAC Permissions Overview

Role	Patient Data	Medical Records	Billing
Doctor	Read details	Read, Create, Update	Read Only
Receptionist	Basic columns only	No Access	No Access
Admin	Full Management	Full Management	Full Management

The matrix above defines the logical access boundaries. These are enforced at the database level using level using MySQL users and specific GRANT statements to ensure Least Privilege.

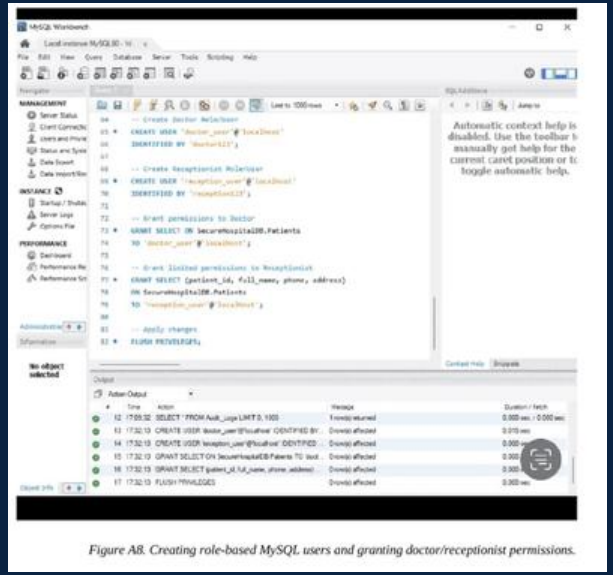

Figure A8. Creating role-based MySQL users and granting doctor/receptionist permissions.

SQL Implementation of User Creation and Privileges

RBAC Implementation (SQL Code)

The Role-Based Access Control is enforced at the database engine engine level using MySQL's native security features:

- **User Creation:** Independent accounts are defined for each role (e.g., doctor_user , reception_user).
- **Privilege Assignment:** The GRANT statement specifies exactly which tables and operations are permitted.
- **Column-Level Security:** Receptionists are restricted to non-medical medical columns in the patients table to protect privacy.

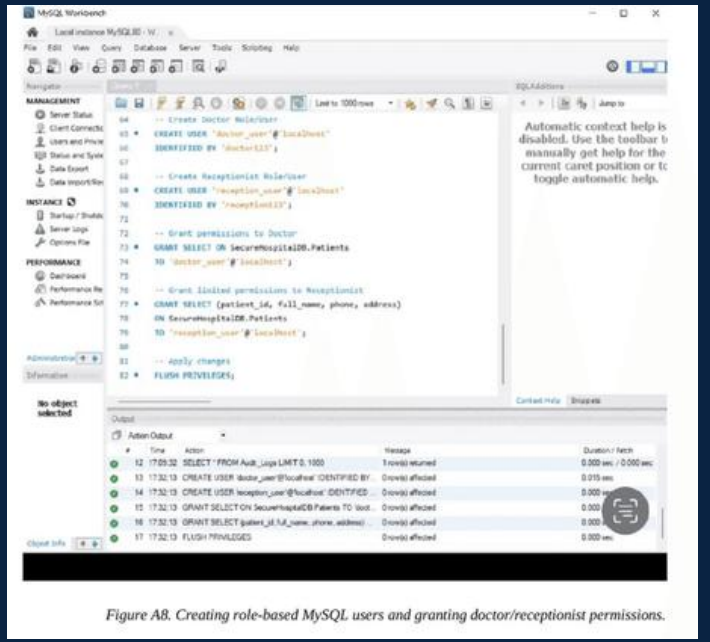

Figure 6: SQL commands for user creation and permission granting

AES Encryption for Medical Data

To protect sensitive patient information, the system implements AES (Advanced (Advanced Encryption Standard), a symmetric encryption algorithm widely widely recognized for its security and efficiency.

Clinical data such as medical history and diagnoses are encrypted before storage using the AES_ENCRYPT function.

> *Engineer's Note: While 'hospital_key' is used here for demonstration, a production-grade implementation would utilize a Key Management (KMS) to rotate and secure encryption keys, ensuring enterprise-level protection.*

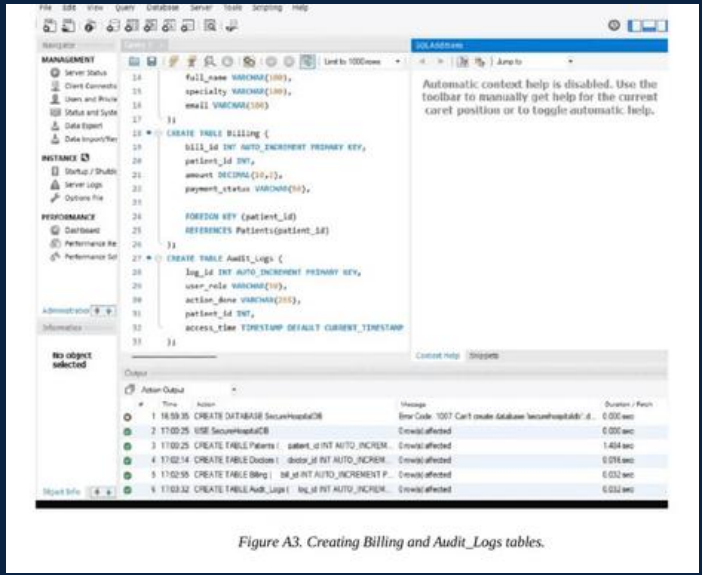

Figure 2: SQL code for creating Billing and Audit_Logs tables

Creating Tables (SQL Code)

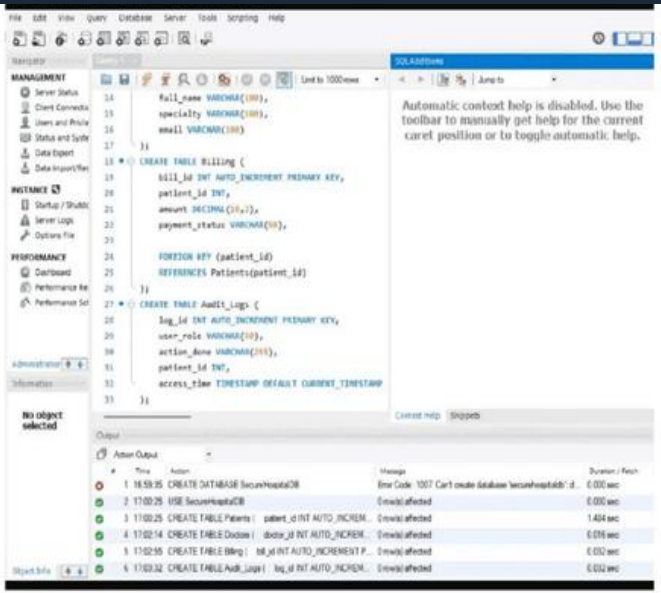

Figure A3. Creating Billing and Audit_Logs tables.

The screenshot above displays the SQL DDL commands used to initialize the **Billing** and **Audit_Logs** tables. These tables are fundamental for tracking financial tracking financial transactions and maintaining a comprehensive record of system activities for security auditing.

Inserting Encrypted Data (SQL Code)

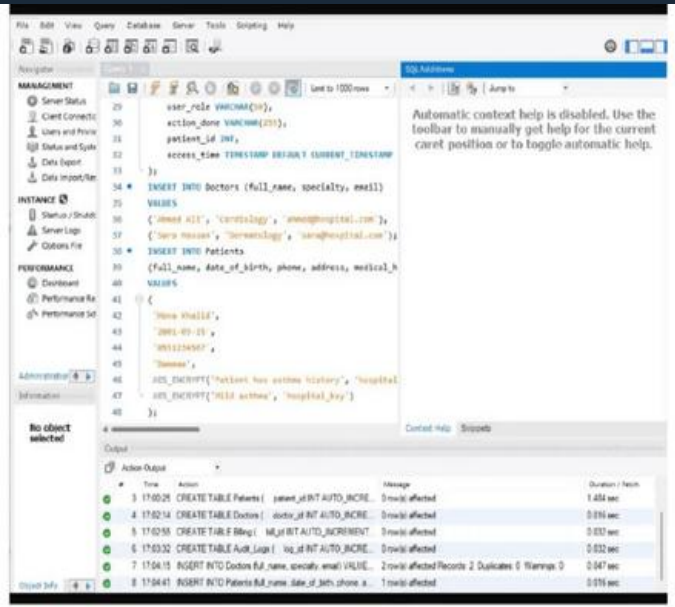

Figure A4. Inserting sample doctors and encrypted patient medical values.

The screenshot above demonstrates the SQL INSERT statements used to populate the database. Sensitive fields such as **medical history** and **diagnoses** are passed through the AES_ENCRYPT function, ensuring that data is secured the moment it enters the system.

Querying Patient Records (SQL Code)

Once data is securely stored, authorized users can retrieve patient patient information using standard SQL queries. This process is process is critical for clinical decision-making and administrative administrative operations.

The screenshot on the right demonstrates a **SELECT** query that that retrieves patient details. Note that sensitive medical handled with care, ensuring that only the necessary exposed to the user based on their **RBAC** permissions.

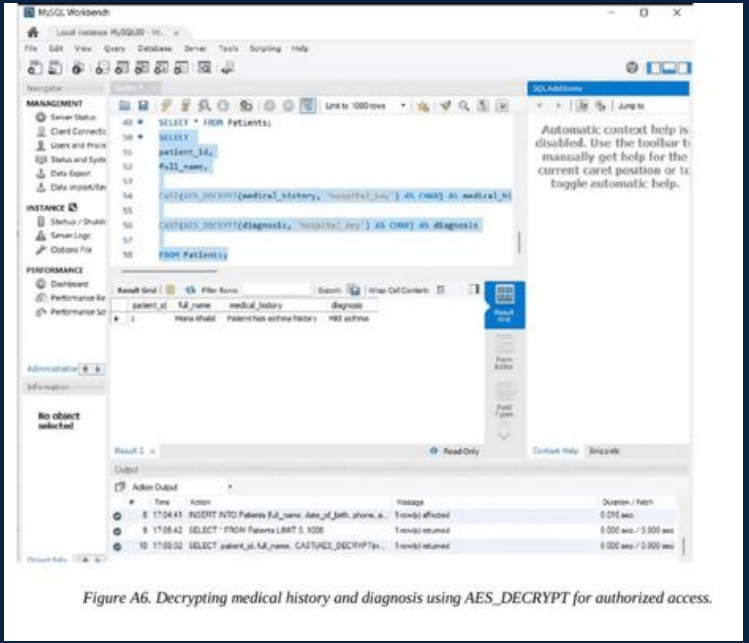

Figure A6. Decrypting medical history and diagnosis using AES_DECRYPT for authorized access.

Figure 4: SQL query results for patient record retrieval

Decrypting Medical Data (SQL Code)

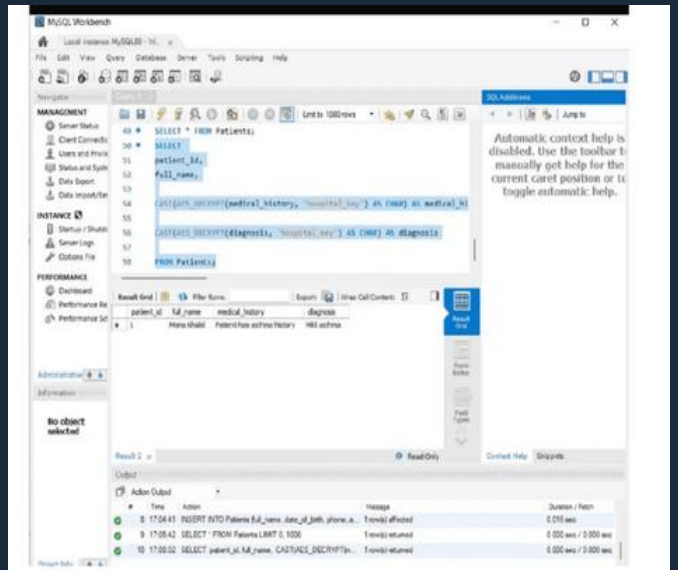

Figure A6. Decrypting medical history and diagnosis using AES_DECRYPT for authorized access.

The screenshot above demonstrates the use of the **AES_DECRYPT** function to retrieve patient medical history and diagnoses. This process ensures that sensitive clinical data is only accessible in plaintext to **authorized personnel** who possess the correct decryption key, maintaining strict confidentiality.

Audit Logging and Accountability

Audit logging is a critical **HIPAA technical safeguard** that requires systems to record and examine activity in information systems that contain or use ePHI.

Our system implements an audit_logs table that automatically captures:

- The **User ID** and Role performing the action.
- The **Action Type** (e.g., View, Insert, Update).
- The **Affected Record** and Table.
- The **Timestamp** and Source IP Address.

This ensures full accountability and provides a trail for security investigations.

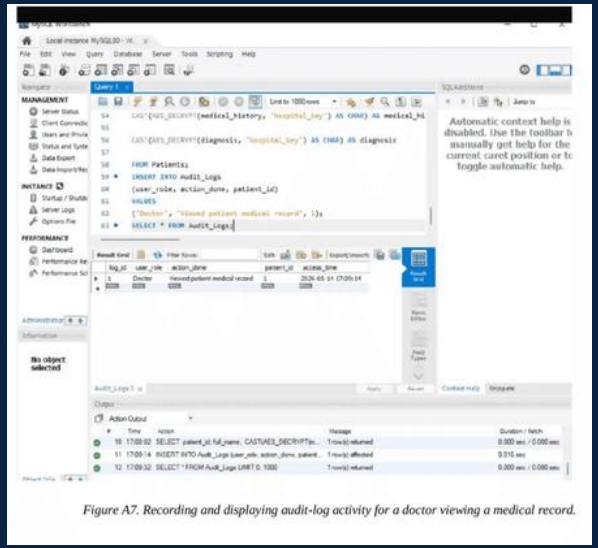

Figure A7. Recording and displaying audit-log activity for a doctor viewing a medical record.

Figure 5: Audit log record showing a doctor viewing a medical record record

Audit Log Activity (SQL Code)

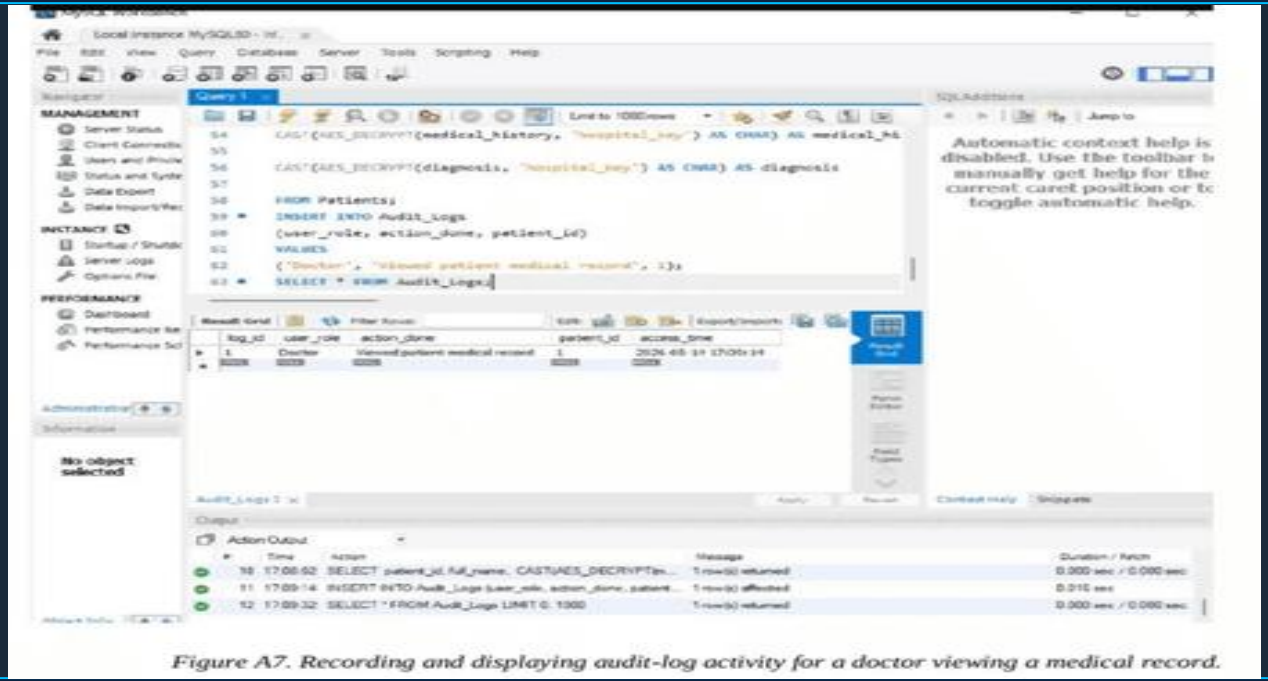

Figure A7. Recording and displaying audit-log activity for a doctor viewing a medical record.

The screenshot above demonstrates the **Audit Log** in action. It records critical details such as the **User Role** (Doctor), the **Action** performed (Viewed patient medical record), the target table, and the **Timestamp**. This level of detail is essential for maintaining a secure and accountable database environment.

Disaster Recovery and Replication

Master Node

Binary Log Stream

Slave Node

IMPLEMENTATION LOGIC

The replication setup is verified by monitoring the **Read_Master_Log_Pos** on the Slave server via the SHOW SLAVE STATUS command. This ensures that the Slave is perfectly synchronized with the Master's binary logs. By tracking the log position, we guarantee that all transactions are replicated in real-time, fulfilling the **HIPAA Contingency Plan** requirement for data backup and rapid disaster recovery.

HIPAA Compliance Analysis

Access Control

The system implements Role-Based Access Control (RBAC) and unique user identification, ensuring that only authorized personnel can access ePHI based on the principle of least privilege.

Audit Controls

Comprehensive audit logs record all access and modifications to patient data, capturing user IDs, actions, and timestamps to support accountability and security reviews.

Integrity

Data integrity is protected through AES encryption and strict modification privileges, preventing unauthorized or accidental alteration of sensitive medical records.

Transmission Security

The database architecture supports secure data exchange and replication, ensuring that ePHI is protected from unauthorized access during transmission and remains available.

Conclusion and Results

The **Secure Hospital Management System** successfully demonstrates a robust technical environment that meets **HIPAA standards**, ensuring patient privacy while maintaining operational efficiency.

INTEGRATED SECURITY FRAMEWORK

Defense-in-Depth Strategy

Our implementation uniquely integrates **AES Encryption**, **Role-Based Access Control (RBAC)**, and **Comprehensive Audit Logging** into a **Logging** into a single, cohesive security framework. This multi-layered approach ensures that sensitive ePHI is protected at every stage—storage, access, and monitoring—providing a level of security that exceeds standard requirements.